

# **Manual d'instal·lació i Configuració**

Arnau Farreras - Shaila Martínez  
1r ASIX

|                                       |           |
|---------------------------------------|-----------|
| <b>Instal·lació del servidor.....</b> | <b>3</b>  |
| <b>Instal·lació de Postgres.....</b>  | <b>6</b>  |
| Repositori.....                       | 6         |
| Cluster.....                          | 7         |
| Verificació.....                      | 8         |
| <b>Configuració SSL.....</b>          | <b>10</b> |
| <b>BACKUPS.....</b>                   | <b>13</b> |
| <b>SERVER SLAVE.....</b>              | <b>15</b> |
| Configuració.....                     | 15        |
| REPLICACIÓ.....                       | 16        |
| <b>RESTAURACIÓ.....</b>               | <b>23</b> |
| <b>PROMOCIÓ SLAVE.....</b>            | <b>25</b> |



Abans de crear els volums físics, els discos NVMe es configuren en RAID 1 per redundància. Això permet que si un disc falla el sistema continua funcionant sense perdre dades.

**Inicialització dels volums físics** per després crear els diferents grups de volums, cada disc té una funció concreta dins del sistema.

```
sudo pvcreate /dev/sdb /dev/sdc /dev/sdd /dev/sde /dev/sdf /dev/sdh
```

### **Grups de volums:**

Cada grup de volums s'ha separat segons la seva funcionalitat, evita que un problema en una part afecti la resta del sistema, per exemple si els logs omplen el disc, la base de dades continuarà funcionant.

vg\_pgdata → dades principals

vg\_pglogs → logs del sistema

vg\_pgbackup → còpies de seguretat

vg\_pgmain → estructura interna del clúster

vg\_pgwal → fitxers WAL

vg\_pgconf → configuració

Comandes:

```
sudo vgcreate vg_pgdata /dev/sdb
sudo vgcreate vg_pglogs /dev/sdc
sudo vgcreate vg_pgbackup /dev/sdd
sudo vgcreate vg_pgmain /dev/sde
sudo vgcreate vg_pgwal /dev/sdf
sudo vgcreate vg_pgconf /dev/sdg
```

### **Creació dels volums lògics:**

Es creen els volums lògics que després es muntaran al sistema. S'utilitza el 70% de l'espai disponible per a una possible ampliació en un futur.

```
sudo lvcreate -l 70%FREE -n lv_pglogs vg_pglogs
sudo lvcreate -l 70%FREE -n lv_pgdata vg_pgdata
sudo lvcreate -l 70%FREE -n lv_pgbackup vg_pgbackup
sudo lvcreate -l 70%FREE -n lv_pgmain vg_pgmain
sudo lvcreate -l 70%FREE -n lv_pgconf vg_pgconf
sudo lvcreate -l 70%FREE -n lv_pgwal vg_pgwal
```

### **Format dels discos**

Tots els volums es formategen en EXT4 perquè és un sistema de fitxers estable i compatible amb Linux i adequat per servidors Postgres.

```
sudo mkfs.ext4 /dev/vg_pglogs/lv_pglogs
sudo mkfs.ext4 /dev/vg_pgdata/lv_pgdata
sudo mkfs.ext4 /dev/vg_pgbackup/lv_pgbackup
sudo mkfs.ext4 /dev/vg_pgmain/lv_pgmain
```



# Instal·lació de Postgres

## Repositori

Aquestes eines són necessàries per afegir el repositori de Postgres i descarregar els paquets correctament:

```
sudo apt install -y curl gnupg2 lsb-release
```

### Afegir la clau GPG del repositori PGDG

S'afegeix la clau oficial de PostgreSQL per garantir que els paquets descarregats siguin legítims.

```
curl -fsSL https://www.postgresql.org/media/keys/ACCC4CF8.asc | \
sudo gpg --dearmor -o /etc/apt/trusted.gpg.d/postgresql.gpg
```

### Afegir el repositori PGDG per a Ubuntu Noble (24.04)

```
echo "deb [signed-by=/etc/apt/trusted.gpg.d/postgresql.gpg] \
https://apt.postgresql.org/pub/repos/apt noble-pgdg main" | \
sudo tee /etc/apt/sources.list.d/pgdg.list
```

## Instal·lació dels components bàsics de postgresql

```
sudo apt update
sudo apt install -y postgresql-common
```

### Evitar la creació automàtica del clúster

Es desactiva la creació automàtica del clúster perquè es vol crear manualment sobre els discos separats d'abans.

```
sudo sed -i 's/#create_main_cluster = true/create_main_cluster = false/'
/etc/postgresql-common/createcluster.conf
```

### Instal·lació final

Instal·lació del servidor, eines client i extensions addicionals.

```
sudo apt install -y postgresql-18 postgresql-client-18 postgresql-contrib-18
```

### Permisos i propietaris

Hem definit els permisos perquè només l'usuari postgres pugui accedir a les dades i als fitxers WAL per millorar la seguretat del sistema.

```
sudo chown -R postgres:postgres /var/lib/postgresql
sudo chown -R postgres:postgres /var/log/postgresql
sudo chown -R postgres:postgres /etc/postgresql
sudo chmod 700 /var/lib/postgresql/wal_real
```

# Cluster

## Configuració del cluster

Configuració de l'idioma i codificació UTF-8 per compatibilitat amb caràcters especials.

```
sudo locale-gen es_ES.UTF-8  
sudo update-locale
```

## Creació del clúster

Es neteja qualsevol estructura creada prèviament abans de generar el clúster definitiu.

```
sudo rm -rf /var/lib/postgresql/18/main/*
```

Creació del clúster utilitzant els discos i estructura preparats d'abans.

```
sudo -u postgres pg_createcluster 18 main \  
--datadir=/var/lib/postgresql/18/main \  
--locale=es_ES.UTF-8 \  
--encoding=UTF8
```

(Aquesta separació es opcional per a la funcionalitat dels wals, tot i així es recomana fer)

## Separació dels WAL

Primer parem el servei del postgresql abans de moure els fitxers.

```
sudo systemctl stop postgresql
```

Entrem com a postgres

```
sudo -u postgres -i
```

Els WAL es mouen a un disc separat per millorar rendiment, reduir colls d'ampolla i augmentar la seguretat de recuperació

```
mv /var/lib/postgresql/18/main/pg_wal/* /var/lib/postgresql/wal_real/
```

```
rmdir /var/lib/postgresql/18/main/pg_wal
```

## Crear enllaç simbòlic

Postgres continuarà accedint als WAL des de pg\_wal, però físicament estaran ubicats en un altre disc.

```
ln -s /var/lib/postgresql/wal_real /var/lib/postgresql/18/main/pg_wal
```

Tornem a encendre el servei:

```
sudo systemctl daemon-reload  
sudo systemctl start postgresql
```

# Verificació

## Verificació del funcionament

`sudo -u postgres psql`

Creem un tablespace separat per comprovar que les dades s'emmagatzemen al disc correcte.

```
CREATE TABLESPACE disco_datos LOCATION '/var/lib/postgresql/data';
```

Crear base de dades: La bd utilitzarà el disc separat destinat a dades.

```
CREATE DATABASE prueba_rendimiento TABLESPACE disco_datos;
```

Proves del funcionament:

Connectarse:  
`\c prueba_rendimiento`

Creem una taula de prova

```
CREATE TABLE usuarios (  
    id SERIAL PRIMARY KEY,  
    nombre TEXT,  
    fecha_registro TIMESTAMP DEFAULT now()  
);
```

Es comprova la codificació UTF-8, funcionament correcte i l'escriptura de dades

```
INSERT INTO usuarios (nombre) VALUES ('Árnau de Sa Palomera');
```

## Comprovacions finals

Verificació del wal que estigui en un disc separat i que funcioni correctament:

```
sudo ls -la /var/lib/postgresql/18/main/pg_wal  
sudo ls -lh /var/lib/postgresql/wal_real
```

Verificació dels logs: els mostra en temps real per detectar possibles errors del servidor.

```
sudo tail -f /var/log/postgresql/postgresql-18-main.log
```

Verificació de que totes les particions i discos estan muntats:

```
df -h | grep -E 'pg|backup|tmp'
```



```

Shaila Oracle VirtualBox
Shaila Project-OJ [Está ejecutando] - Oracle VirtualBox
Shaila Fixer Maquina Visualiza Entrda Dispositivos Ayuda
Shaila admin@hopivibe:~$ sudo systemctl status postgresql.service
Shaila [sudo] password for admini:
Shaila • postgresql.service - PostgreSQL RDBMS
Shaila Loaded: loaded (/usr/lib/systemd/system/postgresql.service; enabled; preset: enabled)
Shaila Active: active (running) since Sat 2024-05-02 13:52:05 UTC; 10min ago
Shaila Process: 1031 ExecStart=/bin/true (code=exited, status=0/SUCCESS)
Shaila Main PID: 1031 (code=exited, status=0/SUCCESS)
Shaila CPU: 19ms
Shaila May 02 13:52:04 hopivibe systemd[1]: Starting postgresql.service - PostgreSQL RDBMS...
Shaila May 02 13:52:05 hopivibe systemd[1]: Finished postgresql.service - PostgreSQL RDBMS.
Shaila admin@hopivibe:~$ sudo tail -f /var/log
Shaila local/ lock/ log/ lost+found/
Shaila admin@hopivibe:~$ sudo tail -f /var/log
Shaila local/ lock/ log/ lost+found/
Shaila admin@hopivibe:~$ sudo tail -f /var/log/postgresql/postgresql-18-main.log
Shaila [sudo] password for admini:
Shaila 2026-05-01 17:56:46.112 UTC [16887] LOG: checkpoint completo: escritos 1007 búfers (6.1%); escritos 3 búfers SLRU; 0 archivos WAL añadidos, 0 eliminados, 0 recic
Shaila lizados; write=100.843 s, sync=0.291 s, total=101.151 s; archivos sincronizados=321, más largo=0.005 s, promedio=0.001 s; distancia=4695 kB, estimación=4695 KB; lsn=0/1BF3190
Shaila 2026-05-01 18:00:04.213 UTC [16887] LOG: empezando checkpoint: time
Shaila 2026-05-01 18:00:04.399 UTC [16887] LOG: checkpoint completo: escritos 1 búfers (0.0%); escritos 0 búfers SLRU; 0 archivos WAL añadidos, 0 eliminados, 0 recicl
Shaila ados; write=0.101 s, sync=0.005 s, total=0.107 s; archivos sincronizados=1, más largo=0.005 s, promedio=0.005 s; distancia=0 kB, estimación=4226 kB; lsn=0/1BF3408
Shaila 2026-05-01 18:01:18.963 UTC [16883] LOG: se recibió petición de apagado rápido
Shaila 2026-05-01 18:01:18.976 UTC [16883] LOG: abortando transacciones activas
Shaila 2026-05-01 18:01:18.980 UTC [16883] LOG: proceso ayudante «logical replication launcher» (PID 16892) terminó con código de salida 1
Shaila 2026-05-01 18:01:18.991 UTC [16887] LOG: apagando
Shaila 2026-05-01 18:01:19.011 UTC [16887] LOG: empezando checkpoint: shutdown immediate
Shaila 2026-05-01 18:01:19.049 UTC [16887] LOG: checkpoint completo: escritos 0 búfers (0.0%); escritos 0 búfers SLRU; 0 archivos WAL añadidos, 0 eliminados, 0 recicl
Shaila ados; write=0.001 s, sync=0.001 s, total=0.003 s; archivos sincronizados=0, más largo=0.000 s, promedio=0.000 s; distancia=0 kB, estimación=3803 KB; lsn=0/1BF34B8
Shaila 2026-05-01 18:01:19.094 UTC [16883] LOG: el sistema de bases de datos está apagado
Shaila c
Shaila admin@hopivibe:~$ df -h | grep -E 'pg|backup|tmp'
Shaila tmpfs          197M    1.2M   196M    1% /run
Shaila tmpfs           985M     0    985M    0% /dev/shm
Shaila tmpfs           5.0M     0    5.0M    0% /run/lock
Shaila /dev/mapper/vg_pgbackup-lv_pgbackup    18G    24K    17G    1% /backup
Shaila /dev/mapper/vg_pgconf-lv_pgconf        3.4G    92K    3.2G    1% /etc/postgresql
Shaila /dev/mapper/vg_system-lv_var            2.0G    94K    1.9G    1% /tmp
Shaila /dev/mapper/vg_main-lv_pgmain          18G    24M    17G    1% /var/lib/postgresql/18/main
Shaila /dev/mapper/vg_data-lv_pgdata         28G    7.7M    26G    1% /var/lib/postgresql/data
Shaila /dev/mapper/vg_wal-lv_pgwal           6.8G    17M    6.5G    1% /var/lib/postgresql/wal_real
Shaila /dev/mapper/vg_logs-lv_pglogs         6.8G    32K    6.5G    1% /var/log/postgresql
Shaila tmpfs          197M    12K   197M    1% /run/user/1000
Shaila admin@hopivibe:~$ hostname
Shaila hopivibe
Shaila admin@hopivibe:~$

```

# Configuració SSL

Generació del certificat SSL amb el seu directori on es guardarà.

```
admini@hopivibe: ~  
admini@hopivibe:~$ sudo mkdir -p /etc/ssl/postgresql  
[sudo] password for admini:  
admini@hopivibe:~$ sudo chown postgres:postgres /etc/ssl/postgresql  
admini@hopivibe:~$ sudo chmod 700 /etc/ssl/postgresql  
admini@hopivibe:~$ hostname  
hopivibe  
admini@hopivibe:~$
```

### Creació del certificat i clau privada:

[illegible][illegible]

### Configuració de permisos:

Shaila Martínez Shaila Martínez Shaila Martínez Shaila Martínez Shaila Martínez Shaila Martínez Shaila Martínez Shaila Martínez Shaila Martínez Shaila Martínez

```
admini@hopivibe: ~  
admini@hopivibe:~$ sudo chmod 600 /etc/ssl/postgresql/server.key  
admini@hopivibe:~$ sudo chmod 644 /etc/ssl/postgresql/server.crt  
admini@hopivibe:~$ hostname  
hopivibe  
admini@hopivibe:~$
```

## Configuració de Postgresql

Això activa el xifrat SSL i indica la ubicació dels certificats.

Shaila Martínez Shaila Martínez Shaila Martínez Shaila Martínez Shaila Martínez Shaila Martínez Shaila Martínez Shaila Martínez Shaila Martínez Shaila Martínez

```
GNU nano 7.2 /etc/postgresql/18/main/postgresql.conf *  
#krb_server_keyfile = 'FILE:${sysconfdir}/krb5.keytab'  
#krb_caseins_users = off  
#gss_accept_delegation = off  
  
# - SSL -  
  
ssl = on  
#ssl_ca_file = ''  
ssl_cert_file = '/etc/ssl/postgresql/server.crt'  
#ssl_crl_file = ''  
#ssl_crl_dir = ''  
ssl_key_file = '/etc/ssl/postgresql/server.key'  
#ssl_ciphers = 'HIGH:MEDIUM:+3DES:!aNULL' # allowed TLSv1.2 ciphers  
#ssl_tls13_ciphers = '' # allowed TLSv1.3 cipher suites, blank for default  
#ssl_prefer_server_ciphers = on  
#ssl_groups = 'X25519:prime256v1'  
#ssl_min_protocol_version = 'TLSv1.2'  
#ssl_max_protocol_version = ''  
#ssl_dh_params_file = ''  
#ssl_passphrase_command = ''  
#ssl_passphrase_command_supports_reload = off  
  
#-----  
# RESOURCE USAGE (except WAL)  
#-----  
  
^G Help      ^O Write Out  ^W Where Is   ^K Cut        ^T Execute    ^C Location   M-U Un  
^X Exit      ^R Read File  ^\ Replace    ^U Paste      ^J Justify    ^/ Go To Line M-E Re
```

Configuració de connexions SSL: obliga a que les connexions remotes utilitzin SSL.

Shaila Martínez Shaila Martínez Shaila Martínez Shaila Martínez Shaila Martínez Shaila Martínez Sha

```
admini@hopivibe: ~
GNU nano 7.2 /etc/postgresql/18/main/pg_hba.conf *

# DO NOT DISABLE!
# If you change this first entry you will need to make sure that the
# database superuser can access the database using some other method.
# Noninteractive access to all databases is required during automatic
# maintenance (custom daily cronjobs, replication, and similar tasks).
#
# Database administrative login by Unix domain socket
local all postgres peer

# TYPE DATABASE USER ADDRESS METHOD

# "local" is for Unix domain socket connections only
local all all peer
# IPv4 local connections:
host all all 127.0.0.1/32 scram-sha-256
# IPv6 local connections:
host all all ::1/128 scram-sha-256
# Allow replication connections from localhost, by a user with the
# replication privilege.
local replication all peer
host replication all 127.0.0.1/32 scram-sha-256
host replication all ::1/128 scram-sha-256
hostssl all all 0.0.0.0/0 scram-sha-256
```

## Reinici del servei

Shaila Martínez Shaila Martínez Shaila Martínez Shaila Martínez Shaila Martínez Shaila Martínez

```
admini@hopivibe: ~$ sudo systemctl restart postgresql
admini@hopivibe:~$ hostname
hopivibe
admini@hopivibe:~$ sudo systemctl status postgresql
● postgresql.service - PostgreSQL RDBMS
   Loaded: loaded (/usr/lib/systemd/system/postgresql.service; enabled; preset: enabled)
   Active: active (exited) since Sat 2026-05-02 17:27:49 UTC; 14s ago
     Process: 1702 ExecStart=/bin/true (code=exited, status=0/SUCCESS)
    Main PID: 1702 (code=exited, status=0/SUCCESS)
       CPU: 9ms

May 02 17:27:49 hopivibe systemd[1]: Stopping postgresql.service - PostgreSQL RDBMS...
May 02 17:27:49 hopivibe systemd[1]: Starting postgresql.service - PostgreSQL RDBMS...
May 02 17:27:49 hopivibe systemd[1]: Finished postgresql.service - PostgreSQL RDBMS.
admini@hopivibe:~$
```

## Verificació SSL

Shaila Martínez Shaila Martínez Shaila Martínez Sl

```
admini@hopivibe: ~$ sudo -u postgres psql
psql (18.3 (Ubuntu 18.3-1.pgdg24.04+1))
Type "help" for help.

postgres=# SHOW ssl;
ssl
-----
on
(1 row)

postgres=#
```

# BACKUPS

## Configuració de les carpetes de backups:

```
sudo mkdir -p /backup/base
sudo mkdir -p /backup/incremental
sudo chown -R postgres:postgres /backup
```

## Configuracions necessaries per al funcionament dels wal per als backups en el postgresql.conf (serà util per a la replica dels nodes):

```
wal_level = replica
archive_mode = on
archive_command = 'test ! -f /var/lib/postgresql/wal_real/%f && cp %p
/var/lib/postgresql/wal_real/%f'
summarize_wal = on #para scripts incrementales
```

## SCRIPT COMPLET

```
#variables
DATA=$(date +%Y-%m-%d)
RUTA_FULL="/backup/base/${DATA}_FULL"
FITXER_RUTA="/backup/base/ultima_ruta.txt"

#crear copia completa de la base de dades a la ruta indicada
sudo -u postgres pg_basebackup -D $RUTA_FULL -Fp -Xs -T
/var/lib/postgresql/data=$RUTA_FULL/extra_data

#modifiquem per a que s'indiqui que aquesta copia es la ultima, així es simple per a fer les
incrementals
echo "$RUTA_FULL" > "$FITXER_RUTA"
```

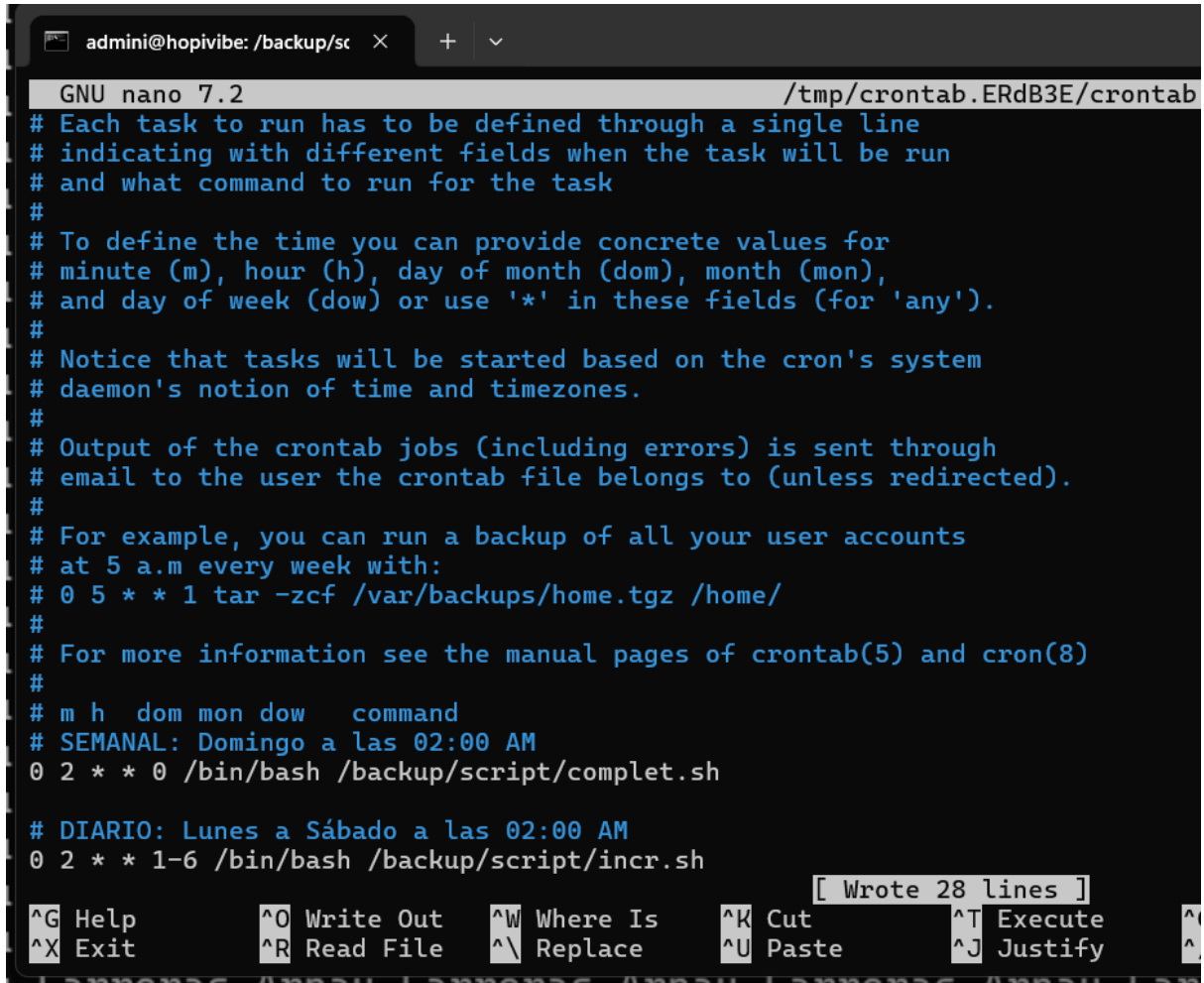
## SCRIPT INCREMENTAL

```
#variables
DATA_AVUI=$(date +%Y-%m-%d)
RUTA_INC="/backup/incremental/${DATA_AVUI}_INC"
FITXER_RUTA="/backup/base/ultima_ruta.txt"

# Llegim la ruta de la última completa des del fitxer
ULTIMA_RUTA=$(cat "$FITXER_RUTA")

# Fem la còpia incremental apuntant directament a la ruta que hem llegit
```

```
sudo -u postgres pg_basebackup -D "$RUTA_INC"
--incremental="${ULTIMA_RUTA}/backup_manifest" -Fp -Xs -v -T
/var/lib/postgresql/data="${RUTA_INC}/extra_data"
```



```
admini@hopivibe: /backup/sc × + ▾
GNU nano 7.2 /tmp/crontab.ERdB3E/crontab
# Each task to run has to be defined through a single line
# indicating with different fields when the task will be run
# and what command to run for the task
#
# To define the time you can provide concrete values for
# minute (m), hour (h), day of month (dom), month (mon),
# and day of week (dow) or use '*' in these fields (for 'any').
#
# Notice that tasks will be started based on the cron's system
# daemon's notion of time and timezones.
#
# Output of the crontab jobs (including errors) is sent through
# email to the user the crontab file belongs to (unless redirected).
#
# For example, you can run a backup of all your user accounts
# at 5 a.m every week with:
# 0 5 * * 1 tar -zcf /var/backups/home.tgz /home/
#
# For more information see the manual pages of crontab(5) and cron(8)
#
# m h dom mon dow   command
# SEMANAL: Domingo a las 02:00 AM
0 2 * * 0 /bin/bash /backup/script/complet.sh

# DIARIO: Lunes a Sábado a las 02:00 AM
0 2 * * 1-6 /bin/bash /backup/script/incr.sh

[ Wrote 28 lines ]
^G Help      ^O Write Out  ^W Where Is   ^K Cut        ^T Execute    ^_
^X Exit      ^R Read File  ^\ Replace    ^U Paste      ^J Justify    ^
```

# SERVER SLAVE

## Configuració

Com planejat anteriorment el servidor es troba en el núvol, sent el AWS, es modificarà algunes dades del sistema, però no es crearan les separacions de discos ni res ja que es prova de com funciona la replicació. Aquesta serà una màquina dins del EC2, ja que incorpora modalitats de Linux a més de poder instal·lar serveis i fer-ho servir com a servidor de replicació.

### **Primer canviem el hostname:**

```
sudo hostnamectl set-hostname hopivibe-slave
```

### **Modifiquem el hostname de /etc/hosts**

```
sudo nano /etc/hosts → 127.0.1.1 hopivibe-slave
```

### **Canviar el ID de la màquina per evitar conflictes.**

```
sudo rm /etc/machine-id  
sudo systemd-machine-id-setup
```

### **Parar el postgres:**

```
sudo systemctl stop postgresql
```

### **Buidem el cluster**

```
sudo rm -rf /var/lib/postgresql/18/main/*
```

### **Fem el slave real:**

```
sudo -u postgres pg_basebackup \  
-h 192.168.1.128 \  
-D /var/lib/postgresql/18/main \  
-U replicador \  
-P \  
-R
```

# REPLICACIÓ

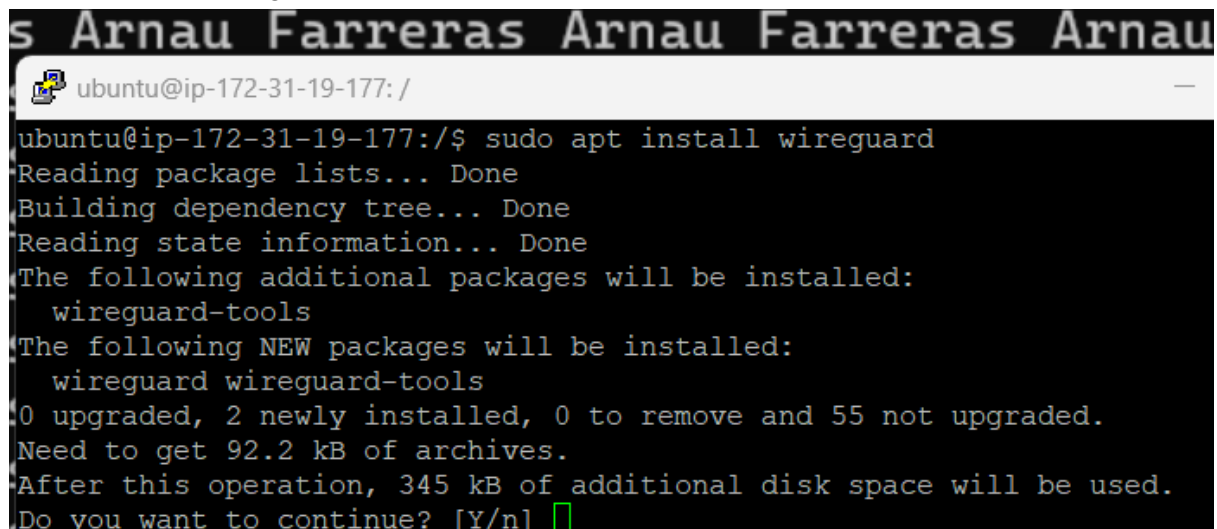
**Primer de tot crearem un usuari de replicació dins del master:**

```
sudo -u postgres psql
```

```
CREATE ROLE replicador  
WITH REPLICATION  
LOGIN  
PASSWORD 'P@ssw0rd1234@';
```

Instal·lar wireguard (per a crear VPN privada entre el servidor master i replica)

Sudo apt install wireguard



```
s Arnau Farreras Arnau Farreras Arnau  
ubuntu@ip-172-31-19-177: /  
ubuntu@ip-172-31-19-177:/$ sudo apt install wireguard  
Reading package lists... Done  
Building dependency tree... Done  
Reading state information... Done  
The following additional packages will be installed:  
  wireguard-tools  
The following NEW packages will be installed:  
  wireguard wireguard-tools  
0 upgraded, 2 newly installed, 0 to remove and 55 not upgraded.  
Need to get 92.2 kB of archives.  
After this operation, 345 kB of additional disk space will be used.  
Do you want to continue? [Y/n]
```

Seguidament en el servidor main i núvol generarem les claus i les observarem, aquestes s'hauran de guardar per més endavant:

```
wg genkey | sudo tee privatekey | wg pubkey | sudo tee publickey
```



```
u Farreras Arnau Farreras A  
Projecte-SQL Master [Corriendo] - Oracle VM VirtualBox  
Archivo Máquina Ver Entrada Dispositivos Ayuda  
admini@hopivibe:~$ sudo cat /etc/wireguard/public.key  
WqtV3wE4bi0QiWhIBNm4JyV6UkmZGxXsaFo8iUu+YA8=  
admini@hopivibe:~$ sudo cat /etc/wireguard/private.key  
YCaJYwui/7mjE0HnpBlvMDkTXsc5sPgYeFdI3raVblo=  
admini@hopivibe:~$
```



```
ubuntu@ip-172-31-19-177: /
ubuntu@ip-172-31-19-177:/$ echo "PUBLICA AWS: $(cat publickey)"
PUBLICA AWS: p+A5r7JNBhoYXByO5PGM23g2vOB/5QFV0Cj9DT1dvik=
ubuntu@ip-172-31-19-177:/$ echo "PRIVADA AWS: $(cat privatekey)"
PRIVADA AWS: qFRSzF2/EMqPKws868ujo6gksS3ys8z/stu8MURbZW0=
ubuntu@ip-172-31-19-177:/$
```

Modificarem el servidor al núvol per a que escolti el servidor master desde la vpn, una ip creada, en la imatge surt 10.0.0.1, per a controlar erros indicar aquesta més la 10.0.0.2:

```
GNU nano 7.2 /etc/postgresql/18/main/postgresql.conf
external_pid_file = '/var/run/postgresql/18-main.pid'
# (change requires restart)

#-----
# CONNECTIONS AND AUTHENTICATION
#-----

# - Connection Settings -

listen_addresses = 'localhost,10.0.0.1' # what IP address(es) to listen on;
# comma-separated list of addresses, defaults to local socket and localhost
```

Afegirem al pg\_hba.conf que permeti la replicació des de l'usuari creat anteriorment.

```
GNU nano 7.2 /etc/postgresql/18/main/pg_hba.conf *
# Noninteractive access to all databases is required during automatic
# maintenance (custom daily cronjobs, replication, and similar tasks).
#
# Database administrative login by Unix domain socket
local all postgres peer

# TYPE DATABASE USER ADDRESS METHOD
# "local" is for Unix domain socket connections only
local all all peer
# IPv4 local connections:
host all all 127.0.0.1/32 scram-sha-256
# IPv6 local connections:
host all all ::1/128 scram-sha-256
# Allow replication connections from localhost, by a user with the
# replication privilege.
local replication all peer
host replication all 127.0.0.1/32 scram-sha-256
host replication all ::1/128 scram-sha-256
host replication replicador 10.0.0.2/32 scram-sha-256

^G Help ^O Write Out ^W Where Is ^K Cut ^T Execute ^C Location
^X Exit ^R Read File ^\ Replace ^U Paste ^J Justify ^_ Go To Line
```

També modifiquem el pg\_hba.conf en el master per a afegir que pugui replicar el servidor slave amb l'usuari indicant des de la ip de la VPN del servidor del núvol, a més de la encriptació:

```
admini@hopivibe: ~  
GNU nano 7.2 /etc/postgresql/18/main/pg_hba.conf *  
  
# DO NOT DISABLE!  
# If you change this first entry you will need to make sure that the  
# database superuser can access the database using some other method.  
# Noninteractive access to all databases is required during automatic  
# maintenance (custom daily cronjobs, replication, and similar tasks).  
#  
# Database administrative login by Unix domain socket  
local all postgres peer  
  
# TYPE DATABASE USER ADDRESS METHOD  
  
# "local" is for Unix domain socket connections only  
local all all peer  
# IPv4 local connections:  
host all all 127.0.0.1/32 scram-sha-256  
# IPv6 local connections:  
host all all ::1/128 scram-sha-256  
# Allow replication connections from localhost, by a user with the  
# replication privilege.  
local replication all peer  
host replication all 127.0.0.1/32 scram-sha-256  
host replication all ::1/128 scram-sha-256  
  
hostssl all all 0.0.0.0/0 scram-sha-256  
  
host replication replicador 10.0.0.1/32 scram-sha-256  
host all all 10.0.0.1/32 scram-sha-256
```

## Configurar Wireguard:

Primer observarem la ip publica del servidor al núvol:

### Resumen de instancia de i-0b23dc0942913c289 (MiPrimerServerAWS) Información

 Conectar

 Estado de la instancia

 Acciones

Se ha actualizado hace less than a minute

**ID de la instancia**  
 i-0b23dc0942913c289

**Dirección IPv6**  
-

**Tipo de nombre de anfitrión**  
Nombre de IP: ip-172-31-19-177.ec2.internal

**Responder al nombre DNS de recurso privado**  
IPv4 (A)

**Dirección asignada automáticamente**  
 98.88.20.83 [IP pública]

**Rol de IAM**  
-

**Dirección IPv4 pública**  
 98.88.20.83 | [dirección abierta](#)

**Estado de la instancia**  
 En ejecución

**Nombre DNS de IP privada (solo IPv4)**  
 ip-172-31-19-177.ec2.internal

**Tipo de instancia**  
t3.micro

**ID de VPC**  
 vpc-0d2b732a0238a258f

**ID de subred**  
 subnet-0880ce960e00d9320

**Direcciones IPv4 privadas**  
 172.31.19.177

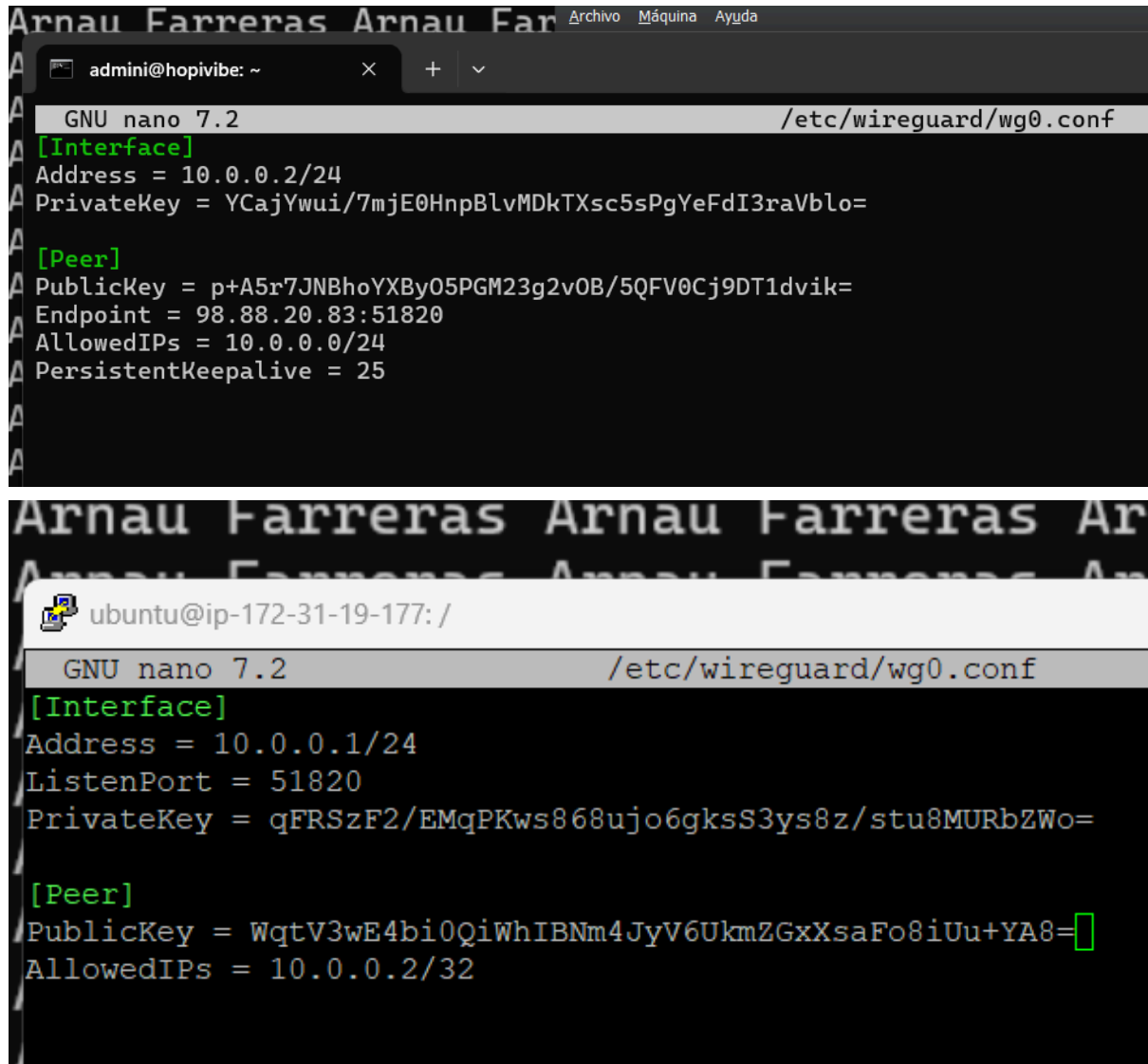
**DNS público**  
 ec2-98-88-20-83.compute-1.amazonaws.com | [dirección abierta](#)

**Direcciones IP elásticas**  
-

**Hallazgo de AWS Compute Optimizer**  
 [Suscribirse a AWS Compute Optimizer para recibir recomendaciones.](#)  
| [Más información](#)

**Nombre del grupo de Auto Scaling**  
-

Crearem un fitxer .conf, wg0.conf que així es com es dirà la interfície de la VPN als dos servidors, la privatekey es la clau privada del propi servidor y la publica la clau publica generada del altre servidor, a més de indicar la ip que tindrà aquest servidor en la vpn a més de indicar la ip publica i port a utilitzar del servidor en el núvol o master. El parametre endpoint només s'ha d'indicar en un dels servidors:



The image consists of two screenshots of terminal windows. The top screenshot shows a terminal window with the title 'admini@hopivibe: ~' and a nano editor editing '/etc/wireguard/wg0.conf'. The configuration for the [Interface] section is: Address = 10.0.0.2/24, PrivateKey = YCajYwui/7mjE0HnpBlvMDkTXsc5sPgYeFdI3raVblo=. The configuration for the [Peer] section is: PublicKey = p+A5r7JNBhoYXBy05PGM23g2v0B/5QFV0Cj9DT1dviK=, Endpoint = 98.88.20.83:51820, AllowedIPs = 10.0.0.0/24, PersistentKeepalive = 25. The bottom screenshot shows a terminal window with the title 'ubuntu@ip-172-31-19-177: /' and a nano editor editing '/etc/wireguard/wg0.conf'. The configuration for the [Interface] section is: Address = 10.0.0.1/24, ListenPort = 51820, PrivateKey = qFRSzf2/EMqPKws868ujo6gksS3ys8z/stu8MURbZW0=. The configuration for the [Peer] section is: PublicKey = WqtV3wE4bi0QiWhIBNm4JyV6UkmZGxXsaFo8iUu+YA8=, AllowedIPs = 10.0.0.2/32.

```
GNU nano 7.2 /etc/wireguard/wg0.conf
[Interface]
Address = 10.0.0.2/24
PrivateKey = YCajYwui/7mjE0HnpBlvMDkTXsc5sPgYeFdI3raVblo=

[Peer]
PublicKey = p+A5r7JNBhoYXBy05PGM23g2v0B/5QFV0Cj9DT1dviK=
Endpoint = 98.88.20.83:51820
AllowedIPs = 10.0.0.0/24
PersistentKeepalive = 25
```

```
GNU nano 7.2 /etc/wireguard/wg0.conf
[Interface]
Address = 10.0.0.1/24
ListenPort = 51820
PrivateKey = qFRSzf2/EMqPKws868ujo6gksS3ys8z/stu8MURbZW0=

[Peer]
PublicKey = WqtV3wE4bi0QiWhIBNm4JyV6UkmZGxXsaFo8iUu+YA8=
AllowedIPs = 10.0.0.2/32
```

Després d'això s'ha de obrir els ports del servidor per a que escolti els missatges del altre servidor.

Un cop això iniciarem la interfície amb:  
sudo wg-quick up <nom del fitxer de configuració>  
I podrem observar la connexió

```
Farreras Arnau Archivo Máquina Ver Entrada Dispositivos Ayuda
ubuntu@ip-172-31-19-177:/
ubuntu@ip-172-31-19-177:/$ sudo wg-quick up wg0
[#] ip link add wg0 type wireguard
[#] wg setconf wg0 /dev/fd/63
[#] ip -4 address add 10.0.0.1/24 dev wg0
[#] ip link set mtu 8921 up dev wg0
ubuntu@ip-172-31-19-177:/$ sudo wg show
interface: wg0
  public key: p+A5r7JNBhoYXByO5PGM23g2vOB/5QFV0Cj9DTldvik=
  private key: (hidden)
  listening port: 51820

peer: WqtV3wE4bi0QiWhIBNm4JyV6UkmZGxXsaFo8iUu+YA8=
  allowed ips: 10.0.0.2/32
ubuntu@ip-172-31-19-177:/$
```

A més de comprovar que es poden fer ping entre ells i en la configuració de internet surt la interfície de la VPN:

```
Arnau Farreras Arnau Farreras Archivo Máquina
ubuntu@ip-172-31-19-177:/
ubuntu@ip-172-31-19-177:/$ ping 10.0.0.2
PING 10.0.0.2 (10.0.0.2) 56(84) bytes of data.
64 bytes from 10.0.0.2: icmp_seq=1 ttl=64 time=161 ms
64 bytes from 10.0.0.2: icmp_seq=2 ttl=64 time=181 ms
^C
--- 10.0.0.2 ping statistics ---
8 packets transmitted, 2 received, 33.333% packet loss, time 1000 ms
rtt min/avg/max/mdev = 160.864/170.998/181.132/10.134 ms
ubuntu@ip-172-31-19-177:/$ ip a
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UP qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
        valid_lft forever preferred_lft forever
    inet6 ::1/128 scope host noprefixroute
        valid_lft forever preferred_lft forever
2: ens5: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 9001 qdisc noqueue state UP qlen 1000
    link/ether 0a:ff:f3:d8:06:b5 brd ff:ff:ff:ff:ff:ff
    inet 172.31.19.177/20 metric 100 brd 172.31.31.255 scope global dynamic
        valid_lft 2469sec preferred_lft 2469sec
    inet6 fe80::8ff:f3ff:fed8:6b5/64 scope link
        valid_lft forever preferred_lft forever
3: wg0: <POINTOPOINT,NOARP,UP,LOWER_UP> mtu 8921 qdisc noqueue state UP qlen 1000
    link/none
    inet 10.0.0.1/24 scope global wg0
        valid_lft forever preferred_lft forever
ubuntu@ip-172-31-19-177:/$

admini@hopivibe:~$ ping 10.0.0.1
PING 10.0.0.1 (10.0.0.1) 56(84) bytes of data.
64 bytes from 10.0.0.1: icmp_seq=1 ttl=64 time=190 ms
64 bytes from 10.0.0.1: icmp_seq=2 ttl=64 time=103 ms
64 bytes from 10.0.0.1: icmp_seq=3 ttl=64 time=101 ms
64 bytes from 10.0.0.1: icmp_seq=4 ttl=64 time=105 ms
^C
--- 10.0.0.1 ping statistics ---
4 packets transmitted, 4 received, 0% packet loss, time 100.990 ms
rtt min/avg/max/mdev = 100.990/124.610/189.737/37.622 ms
admini@hopivibe:~$ ip a
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UP qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
        valid_lft forever preferred_lft forever
    inet6 ::1/128 scope host noprefixroute
        valid_lft forever preferred_lft forever
2: enp0s3: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc noqueue state UP qlen 1000
    link/ether 08:00:27:3d:f4:35 brd ff:ff:ff:ff:ff:ff
    inet 192.168.1.128/24 brd 192.168.1.255 scope global dynamic
        valid_lft forever preferred_lft forever
    inet6 fe80::a00:27ff:fe3d:f435/64 scope link
        valid_lft forever preferred_lft forever
3: enp0s8: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc noqueue state UP qlen 1000
    link/ether 08:00:27:4f:00:80 brd ff:ff:ff:ff:ff:ff
    inet 192.168.1.19/24 metric 100 brd 192.168.1.255 scope global dynamic
        valid_lft 85204sec preferred_lft 85204sec
    inet6 fe80::a00:27ff:fe4f:80/64 scope link
        valid_lft forever preferred_lft forever
4: wg0: <POINTOPOINT,NOARP,UP,LOWER_UP> mtu 1420 qdisc noqueue state UP qlen 1000
    link/none
    inet 10.0.0.2/24 scope global wg0
        valid_lft forever preferred_lft forever
admini@hopivibe:~$
```

Un cop això replicarem la base de dades amb l'usuari creat anteriorment:

```
Arnau Farreras Arnau Farreras Arnau Farreras Arnau
ubuntu@ip-172-31-19-177: /
ubuntu@ip-172-31-19-177:/$ sudo -u postgres pg_basebackup -h 10.0.0.2 -D /var/lib/postgresql/18
/main/ -U replicador -P -R
Password:
17160/31572 kB (54%), 1/2 tablespaces
```

I com es un servidor de estats units, haurem de generar el UTF-8 en espanyol, que es el que utilitzem en el nostre servidor master:

```
Arnau Farreras Arnau Farreras Arnau
ubuntu@ip-172-31-19-177: /
ubuntu@ip-172-31-19-177:/$ sudo locale-gen es_ES.UTF-8
Generating locales (this might take a while)...
  es_ES.UTF-8... done
Generation complete.
ubuntu@ip-172-31-19-177:/$ sudo update-locale
ubuntu@ip-172-31-19-177:/$ sudo systemctl restart postgresql
ubuntu@ip-172-31-19-177:/$ sudo -u postgres psql
psql (18.3 (Ubuntu 18.3-1.pgdg24.04+1))
Type "help" for help.

postgres=#
```

Fent un `SELECT pg_is_in_recovery();` podem comprovar si el servidor esta en mode replicació, si surt l'indicació "t":

```
reras Arnau Farreras Ar
ubuntu@ip-172-31-19-177: /
postgres=# SELECT pg_is_in_recovery();
 pg_is_in_recovery
-----
 t
(1 row)

postgres=#
```

Y es pot comprovar si funciona inserint dades on sigui i apareixeran de seguida dins del servidor de replica:

```
as Arnau Farreras Arnau Farreras Arnau Farreras A

admini@hopivibe: ~
postgres=# insert into test_replica (nom)
VALUES ('hola')
;
INSERT 0 1
postgres=# \q
admini@hopivibe:~$ hostname
hopivibe
admini@hopivibe:~$

ubuntu@ip-172-31-19-177: /
postgres=# SELECT * FROM test_replica;
 id |      nom
----+-----
  1 | ReplicaOK
  2 | AWS
  3 | hola
(3 rows)

postgres=# \q
ubuntu@ip-172-31-19-177:/$ hostname
ip-172-31-19-177
ubuntu@ip-172-31-19-177:/$
```

A més en el de replica no deixarà afegir-ne dades:

```
u Farreras Arnau Farreras Arnau F

ubuntu@ip-172-31-19-177: ~
postgres=# CREATE TABLE papa(hola INT PRIMARY KEY);
ERROR:  cannot execute CREATE TABLE in a read-only transaction
postgres=#
```

Altres configuracions, com a slave si el master cau o per recuperar dades estaria interessant tenir aquesta informació:

Crear fitxer standby.signal

```
sudo -u postgres touch /var/lib/postgresql/18/main/standby.signal
```

Editar postgresql.auto.conf

```
sudo nano /var/lib/postgresql/18/main/postgresql.auto.conf
```

```
primary_conninfo = 'host=192.168.1.128 port=5432 user=replicador
```

```
password=P@ssw0rd1234@ dbname=postgres'
```

```
Shaila
Shaila
Shaila
Shaila
Shaila
Projecte-SQL Slave [S'està executant] - Oracle VirtualBox
Fitxer  Màquina  Visualitza  Entrada  Dispositius  Ajuda
GNU nano 7.2 /var/lib/postgresql/18/main/postgresql.auto.conf
# Do not edit this file manually!
# It will be overwritten by the ALTER SYSTEM command.
primary_conninfo = 'host=192.168.1.128 port=5432 user=replicador password=P@ssw0rd1234@ dbname=postgres'
```

Permisos

```
sudo chown postgres:postgres /var/lib/postgresql/18/main/postgresql.auto.conf
```

# RESTAURACIÓ

Creació del script:

```
sudo nano /usr/local/bin/restauracio_completa.sh
```

```
#!/bin/bash
```

```
DB_PATH="/var/lib/postgresql/18/main"
```

```
BASE_DIR="/backup/base"
```

```
INCREMENTAL_DIR="/backup/incremental"
```

```
WAL_DIR="/var/lib/postgresql/wal_real"
```

```
LATEST_BASE=$(ls -td $BASE_DIR/* | head -1)
```

```
LATEST_INCREMENTAL=$(ls -td $INCREMENTAL_DIR/* | head -1)
```

```
echo "parant postgres"
```

```
sudo systemctl stop postgresql
```

```
echo "eliminant dades antigues"
```

```
sudo rm -rf ${DB_PATH:?}/*
```

```
echo "restaurant ultima copia completa"
```

```
sudo cp -a $LATEST_BASE/* $DB_PATH/
```

```
echo "restaurant ultima copia incremental"
```

```
sudo cp -a $LATEST_INCREMENTAL/* $DB_PATH/
```

```
echo "configurant permisos"
```

```
sudo chown -R postgres:postgres $DB_PATH
```

```
echo "activant recovery mode"
```

```
sudo -u postgres touch $DB_PATH/recovery.signal
```

```
echo "configurant restauracio wal"
```

```
sudo bash -c "cat >> $DB_PATH/postgresql.auto.conf <<EOF
```

```
restore_command = 'cp $WAL_DIR/%f %p'
```

```
EOF"
```

```
echo "iniciant postgresql"
```

```
sudo systemctl start postgresql
```

```
echo "restauracio completada"
```

```
Shaila Projecte-SQL Master (Captura 1) [S'està executant] - Oracle VirtualBox
Shaila Fitxer Màquina Visualitza Entrada Dispositius Ajuda
Shaila GNU nano 7.2 /usr/local/bin/restauracio_completa.sh
Shaila #!/bin/bash
Shaila DB_PATH="/var/lib/postgresql/18/main"
Shaila BASE_DIR="/backup/base"
Shaila INCREMENTAL_DIR="/backup/incremental"
Shaila WAL_DIR="/var/lib/postgresql/wal_real"
Shaila LATEST_BASE=$(ls -td $BASE_DIR/* | head -1)
Shaila LATEST_INCREMENTAL=$(ls -td $INCREMENTAL_DIR/* | head -1)
Shaila echo "parant postgres"
Shaila sudo systemctl stop postgresql
Shaila echo "eliminant dades antigues"
Shaila sudo rm -rf ${DB_PATH:?}/*
Shaila echo "restaurant ultima copia completa"
Shaila sudo cp -a $LATEST_BASE/* $DB_PATH/
Shaila echo "restaurant ultima copia increemental"
Shaila sudo cp -a $LATEST_INCREMENTAL/* $DB_PATH/
Shaila echo "configurant permisos"
Shaila sudo chown -R postgres:postgres $DB_PATH
Shaila echo "recovery mode"
Shaila sudo -u postgres touch $DB_PATH/recovery.signal
Shaila echo "configurant restauracio wal"
Shaila sudo bash -c "cat >> $DB_PATH/postgresql.auto.conf <<EOF
Shaila restore_command = 'cp $WAL_DIR/%f %p'
Shaila EOF"
Shaila echo "iniciant postgresql"
Shaila sudo systemctl start postgresql
Shaila echo "restauracio completada"
```

Donar permisos:

```
sudo chmod +x /usr/local/bin/restauracio_completa.sh
```

Execució:

```
sudo /usr/local/bin/restauracio_completa.sh
```

#### Funcionament del script complet:

Permet realitzar una restauració completa de postgres utilitzant l'últim backup complet, l'últim backup incremental i tots els arxius WAL disponibles.

Primer para el postgres perquè no es puguin modificar els fitxers mentre la bd està en funcionament, després elimina les dades antigues del cluster. Restaura l'últim backup complet de la bd que conté tota l'estructura, configuració, dades i taules, restaura l'últim backup incremental més recent després del complet.

Es configura els permisos perquè es pugui accedir als fitxers restaurats i activa el mode recovery.

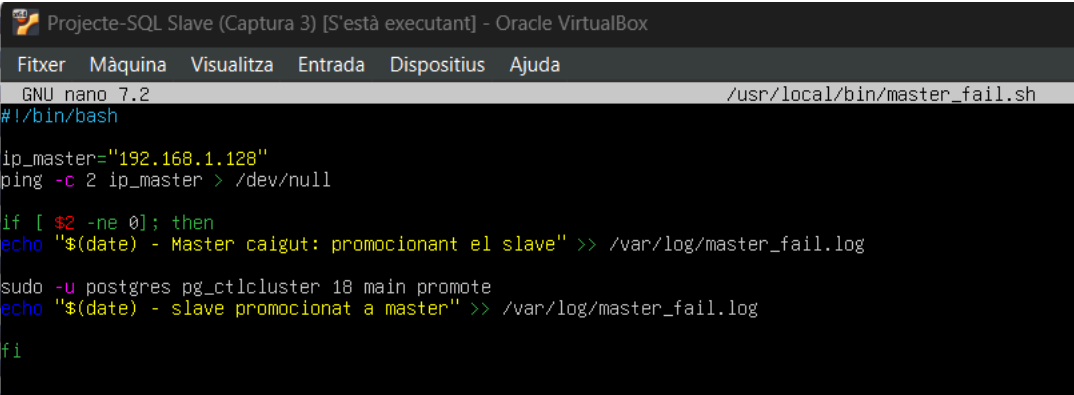
Postgres aplica automàticament els fitxers WAL per reconstruir l'estat més recent de la bd, que contenen les transaccions fetes després del backup incremental.

Finalment inicia el postgres.



# PROMOCIÓ SLAVE

Script:



```
Projecte-SQL Slave (Captura 3) [S'està executant] - Oracle VirtualBox
Fitxer  Màquina  Visualitza  Entrada  Dispositius  Ajuda
GNU nano 7.2 /usr/local/bin/master_fail.sh
#!/bin/bash
ip_master="192.168.1.128"
ping -c 2 ip_master > /dev/null
if [ $? -ne 0 ]; then
echo "$(date) - Master caigut: promocionant el slave" >> /var/log/master_fail.log
sudo -u postgres pg_ctlcluster 18 main promote
echo "$(date) - slave promocionat a master" >> /var/log/master_fail.log
fi
```

Donar permisos:

```
sudo chmod +x /usr/local/bin/master_fail.sh
```

Funcionament:

El script fa ping al master, si no respon executa “pg\_ctlcluster 18 main promote”. El postgres deixa recovery mode, accepta escriptures i es converteix en el nou master.

Es podria comprovar amb “SELECT pg\_is\_in\_recovery()” al slave i si surt “t” significa que està en slave abans de la caiguda del master, si després surt “f” vol dir que està acuant de master.

En una cas real es podria programar una tasca al cron que comprovi el master a cada minut i si cau, el slave es promociona automàticament.